

PowerShell Scripting



PowerShell Scripting

- Log Analysis with Power Shell
- Task Automation with powershell
- Powershell for offensive security



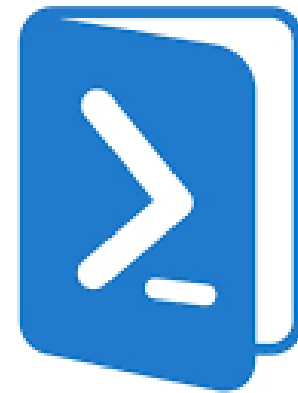
Labs

- Students will prepare an incident report based on the facts and logs provided to them.



Log Analysis with PowerShell

- **PowerShell** is a powerful, native tool on Windows for **collecting, filtering, correlating, and investigating logs** during security monitoring, incident response, and threat hunting. It is widely used by SOC analysts and incident responders.
- **Why Use PowerShell for Log Analysis**
 - Native to Windows (no additional tools required)
 - Direct access to **Windows Event Logs**
 - Fast filtering and automation
 - Ideal for **incident response and forensic triage**
 - Can be integrated with SIEM workflows



PowerShell



Log Sources Analyzed Using PowerShell

- Windows Event Logs
 - Security
 - System
 - Application
- PowerShell Operational Logs
- Firewall logs
- Sysmon logs
- IIS and application logs

PowerShell Operational Logs

- Event Viewer →
- Applications and Services Logs →
- Microsoft →
- Windows →
- PowerShell

Basic Windows Event Log Analysis

```
PS C:\> Get-WinEvent -ListLog *
```

LogMode	MaximumSizeInBytes	RecordCount	LogName
-----	-----	-----	-----
Circular	15728640	10112	Windows PowerShell
Circular	1052672	0	Visual Studio
Circular	20971520	38373	System
Circular	20971520	31622	Security
Circular	1052672	3540	OneApp_IGCC
Circular	1052672	2536	0Alerts
Circular	20971520	0	Key Management Service
Circular	1052672	0	Internet Explorer
Circular	4194304	345	HP Diagnostics
Circular	1052672	1219	HP Analytics
Circular	20971520	0	HardwareEvents
Circular	20971520	27646	Application



Retrieve Recent Security Events

```
PS C:\> Get-WinEvent -LogName Security -MaxEvents 20 | ft -AutoSize -Wrap

ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated          Id LevelDisplayName Message
-----
03-Jan-26 10:33:01 PM 5379 Information Credential Manager credentials were read.

Subject:
  Security ID:          S-1-5-21-1341496841-55697840-1495310029-1001
  Account Name:         Jeetu
  Account Domain:       WORKSTATION
  Logon ID:              0x4682F

03-Jan-26 10:33:01 PM 5379 Information Credential Manager credentials were read.

Security ID:          S-1-5-21-1341496841-55697840-1495310029-1001

Account Domain:       WORKSTATION

Logon ID:              0x4682F
```



PowerShell Attack Detection

```
Untitled1.ps1* X
1  cls
2  Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |
3  where-Object {$_.Id -eq 4104} |
4  select TimeCreated, Message
5
```

TimeCreated	Message
03-Jan-26 10:36:12 PM	Creating Scriptblock text (1 of 1):...
03-Jan-26 10:36:12 PM	Creating Scriptblock text (1 of 1):...
03-Jan-26 1:51:50 PM	Creating Scriptblock text (1 of 1):...
27-Dec-25 1:26:01 PM	Creating Scriptblock text (1 of 1):...
25-Dec-25 3:09:34 PM	Creating Scriptblock text (1 of 1):...
25-Dec-25 3:09:26 PM	Creating Scriptblock text (1 of 1):...
25-Dec-25 3:08:53 PM	Creating Scriptblock text (1 of 1):...
25-Dec-25 3:07:02 PM	Creating Scriptblock text (1 of 1):...



Suspicious Network Connections

```
Untitled1.ps1* X
1  cls
2  Get-NetTCPConnection |
3  Where-Object {$_.State -eq "Established"} |
4  Select LocalAddress, LocalPort, RemoteAddress, RemotePort
5
```

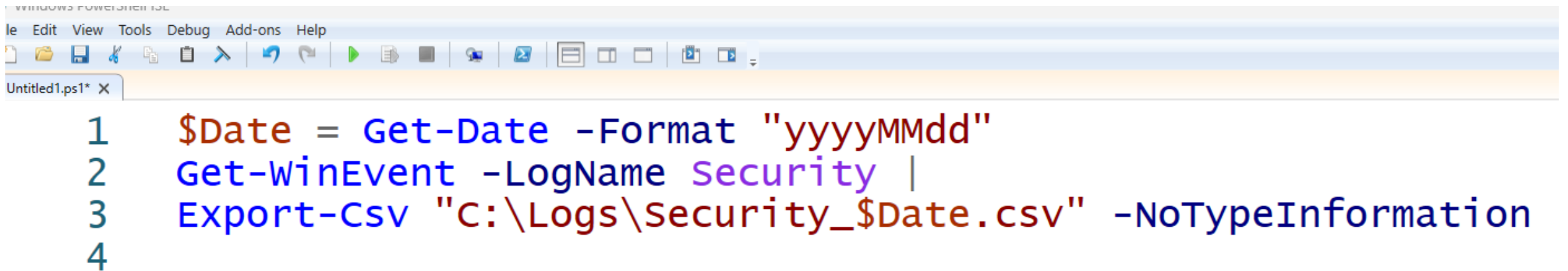
LocalAddress	LocalPort	RemoteAddress	RemotePort
192.168.1.5	63142	23.55.106.59	443
192.168.1.5	62716	104.18.39.21	443
192.168.1.5	59569	74.125.200.113	443
192.168.1.5	58979	23.55.106.59	443
192.168.1.5	58470	172.66.2.5	80
192.168.1.5	58194	192.168.1.102	8009
192.168.1.5	56243	172.64.155.209	443
192.168.1.5	54137	4.213.25.240	443
192.168.1.5	53109	192.168.1.102	8009
192.168.1.5	50203	192.168.1.100	8008
192.168.1.5	50201	192.168.1.102	8008
192.168.1.5	50199	74.125.200.188	5228
127.0.0.1	49674	127.0.0.1	49673
127.0.0.1	49673	127.0.0.1	49674



Task Automation with powershell

- Log collection and parsing
- User and access management
- Incident response actions
- System hardening checks
- Compliance and security audits
- Threat hunting routines

Automating Log Collection



The image shows a screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The window has a menu bar with 'File', 'Edit', 'View', 'Tools', 'Debug', 'Add-ons', and 'Help'. Below the menu bar is a toolbar with various icons for file operations, editing, and execution. The main editing area shows a PowerShell script with four lines of code, each preceded by a line number (1, 2, 3, 4). The script is designed to collect security logs and export them to a CSV file.

```
1 $Date = Get-Date -Format "yyyyMMdd"
2 Get-WinEvent -LogName Security |
3 Export-Csv "C:\Logs\Security_$Date.csv" -NoTypeInfoation
4
```



Scheduled Task Automation

Untitled1.ps1* X

```
1  cls
2
3  $action = New-ScheduledTaskAction -Execute "PowerShell.exe" `
4  -Argument "-File C:\Scripts\SecurityCheck.ps1"
5
6  $trigger = New-ScheduledTaskTrigger -Daily -At 2am
7
8  Register-ScheduledTask -TaskName "DailySecurityCheck" `
9  -Action $action -Trigger $trigger
10
11
```



Best Practices for PowerShell Automation

- Use signed scripts (Execution Policy)
- Follow least privilege
- Add logging and error handling
- Test in staging before production
- Secure scripts and credentials
- Document automation workflows

Powershell for offensive security

- PowerShell is a dual-use technology. In offensive security, it is used by red teams and penetration testers to simulate real attacker behavior for authorized security assessments.
- The focus is on emulation, enumeration, and proof-of-control, not uncontrolled exploitation.
- Attackers and red teams prefer PowerShell because it is:
 - Native to Windows (no additional binaries required)
 - Trusted and often whitelisted
 - Extremely powerful for system interaction
 - Ideal for Living-off-the-Land (LotL) techniques
 - Scriptable and memory-resident



Offensive Security Use Cases of PowerShell

- **Reconnaissance & Enumeration** – Used to understand the environment after initial access.
- **Credential & Identity Awareness (Simulation)** – PowerShell helps simulate identity-focused attacks without dumping real credentials.
- **Living-off-the-Land Techniques (LOLBins)** – PowerShell enables attackers to operate using built-in Windows utilities.
- **Lateral Movement Simulation** - PowerShell is used to emulate movement across systems in a domain.
- **Command & Control (C2) Emulation** - Red teams use safe PowerShell callbacks to test detection.
- **Persistence Testing** – PowerShell can emulate persistence mechanisms.



Role of PowerShell in Red, Blue & Purple Teams

- **Red Team:** Simulates realistic attacker behavior
- **Blue Team:** Detects and responds to PowerShell misuse
- **Purple Team:** Tunes detections based on observed gaps





That's all Folks!